

Learning to Herd Under Limited Visibility: Recurrent PPO with Procedural Curricula for Single-Agent Shepherd

Author One

Department of Computer Science
University Name
City, Country
author.one@university.edu

Author Two

Department of Computer Science
University Name
City, Country
author.two@university.edu

Author Three

Department of Computer Science
University Name
City, Country
author.three@university.edu

Abstract—Shepherding asks one agent to steer a flock of reactive agents into a goal region. We trace four versions of a learned herder with limited visibility. The first two solve their own task (99% and 69% success) but fail once the goal moves, and a recurrent policy trained on four fixed layouts failed on held-out maps. The final version trains recurrent PPO on procedurally generated layouts with domain randomization and an adaptive curriculum. Over three seeds per curriculum and 150 held-out episodes per scenario, it delivers 30% to 32% of the flock on a procedural test suite, against 47% for a collect-and-drive heuristic that sees the full obstacle map. It is ahead of the heuristic on average on some hand-designed maps, and the curriculum decides which: a split field under the base curriculum, and a cluttered map, corridors and gates under a curriculum that trains longer at full difficulty. Results on single maps vary by up to 39 points between seeds, against at most 8 on the procedural test. The learned policies also bring up to 79% of the flock into the goal on average but end with 21% to 48%, because the reward does not pay for holding the flock there.

Index Terms—Shepherding, Reinforcement learning, Partial observability, Curriculum learning, Domain randomization, Generalization

I. INTRODUCTION

Shepherding is the problem of steering a group of interacting agents into a target region with one or a few active agents [1], [2]. The herder never carries a sheep; it moves the flock by choosing where to stand. The same structure appears in livestock robotics [3], crowd guidance, and the indirect control of robot swarms [4], [5].

Strömbom *et al.* showed that a short rule reproduces the GPS traces of working sheepdogs: collect the furthest sheep when the flock is spread out, otherwise drive it from behind toward the goal [2]. The rule assumes the dog sees the whole flock and that the field is open. Both assumptions fail when visibility is limited, obstacles block the direct route, or the flock splits. A learned policy could adapt to such conditions without hand tuning, but the learning problem is a partially observable Markov decision process (POMDP) with a long horizon, many interacting bodies, and sparse success.

This paper reports what it took to get a learned herder that works on maps it has not seen, and how far it still is from the heuristic. The first three versions of our system each solved the setting they were trained on and failed when it

changed, as Cobbe *et al.* found for RL trained on a fixed set of levels [6]. The fourth trains on procedurally generated layouts, and we test it on held-out procedural layouts and five hand-designed maps.

Our contributions are:

- A history of four herding environments and policies, with each version’s failure measured on 150 or more episodes where the model still exists (Section III).
- A recurrent PPO herder trained on procedural layouts with an adaptive curriculum, and the four failure modes we had to remove from the curriculum and checkpointing pipeline (Section V, Section VII.E).
- An evaluation over three seeds for each of two curriculum settings, against a collect-and-drive heuristic and a behavioral clone, with bootstrap confidence intervals (Section VII).
- Evidence that single hand-designed maps are an unreliable test, and that the learned policies deliver much more of the flock than they keep (Section VII).

II. RELATED WORK

Heuristic shepherding. Reynolds’ boids model flocking as local attraction, alignment and repulsion [7]. Lien *et al.* described geometric herding behaviors for motion planning [1], and Vaughan *et al.* herded ducks with a mobile robot [4]. The collect-and-drive algorithm of Strömbom *et al.* [2] and its robotic extensions [8] are the standard single-herder baseline. Control-theoretic analyses [5] and a review of biologically inspired robot herding [3] complete this line of work.

Learning to shepherd. Hussein *et al.* learn collect and drive behaviors with curriculum PPO [9]. Hasan *et al.* treat cooperative multi-shepherd herding as payload protection [10], and Napolitano *et al.* learn decentralized policies for several herders [11]. These works usually assume wider sensing than ours, or split the task into target assignment and single-target driving.

Generalization and evaluation. Recurrent policies are the usual response to partial observability [12], [13]. Domain randomization [14] and curricula [15] widen the training distribution, and held-out procedural levels are the standard test of whether a policy generalizes [6]. Agarwal *et al.* show

that results from few runs are often unreliable [16], so we report bootstrap intervals [17] and per-seed values. Behavioral cloning [18], [19] is the simplest imitation baseline, and its compounding error under distribution shift is well known [20].

III. DEVELOPMENT HISTORY

Table I summarizes the four versions. All use a 20×20 field, ten sheep, a dog that moves one unit per step, and success when every sheep is within 2 units of the goal.

The first version (v1) gave the dog full visibility, had no obstacles, and fixed the goal at (18, 18). Its observation was the dog’s position and each sheep’s offset from the dog, with no goal. PPO solved this task in 99% of episodes. When we draw the goal uniformly from $[2.4, 17.6]^2$ instead, success falls to 2%, below a random policy’s 4%, because the policy has only learned to herd toward one corner. The second version (v2) limited visibility to a radius of 8, marking unseen sheep with a sentinel value, and added two fixed obstacles. PPO succeeded in 69% of episodes with the fixed goal and in 3% with a random one.

The third version used the current simulator with a recurrent policy, four fixed obstacle layouts, and the flock always spawned in the corner opposite the goal. It succeeded in 75% of training-layout episodes but in none of the episodes on two held-out maps (split field and open field, Section IV.C), where the heuristic succeeded in 25% and 100%. These numbers come from a small evaluation of one seed, and the model no longer exists to re-evaluate. Each version removed one source of overfitting and exposed the next.

IV. ENVIRONMENT

A. Flock and dog

A dog at position d_t herds N sheep s_t^i toward a goal g among at most eight axis-aligned rectangles. The action $a_t \in [-1, 1]^2$ is normalized to unit length and clipped to the longest collision-free part of the step. An episode succeeds when every sheep is within $\rho = 2$ of g and is truncated after 700 steps. Each sheep moves at constant speed v_s along a sum of forces [2], [7]. It flees the dog within a flee radius R_f , is pulled toward the flock centroid with gain α and toward a leader sheep with gain β , and is repelled by neighbors within

R_r , and by nearby obstacles. Gaussian noise ($\sigma = 0.02$) is added, and a sheep inside the flee radius also receives a small pull of 0.03 toward the goal. Nominal values are $v_s = 0.32$, $R_f = 5.5$, $\alpha = 0.07$, $R_r = 1.0$ and $\beta = 0.04$.

B. Observation

The dog sees sheep within a Euclidean radius R_v (nominally 7.5); obstacles do not block sight. The policy receives a 98-dimensional egocentric vector with no absolute position. It holds the direction and distance to the goal, the distances to the arena walls, the previous action, and statistics of the visible sheep (fraction visible, centroid offset, mean and maximum distance, spread). A memory block holds the offset to the last seen flock centroid, the time since it was seen, and its direction and distance to the goal, followed by the remaining time. Then come 16 sheep slots of (relative x , relative y , visible flag), with hidden sheep and unused slots set to zero, and 24 lidar ranges to obstacles and walls. Sheep positions and executed actions carry Gaussian noise.

C. Scenarios and splits

Layouts come from five topologies: open, scattered blobs, corridors of parallel walls, a gate (one wall across the field with an opening), and short bars that split the flock (Fig. 1). Goals are uniform in $[2.4, 17.6]^2$, the flock spawns at least 9 units from the goal with the dog behind it, and a flood fill rejects layouts in which the goal is unreachable.

Train samples topologies, dynamics and noise from training ranges that the curriculum narrows (Section V.C). *Validation* uses the full training ranges with its own seeds and selects checkpoints. *Test* uses wider ranges that extend past training at both ends, on unseen seeds: 8 to 16 sheep instead of 6 to 14, gates 2.2 to 4.0 units wide instead of 3.0 to 6.0, more blobs, longer walls, visibility radii from 4.5 to 10.0 instead of 5.0 to 9.5, and wider dynamics and noise ranges. Five hand-designed maps with fixed parameters and 10 sheep give further held-out cases. *Corridor* has two staggered pairs of walls, *dense* has five large blobs near the start, *narrow gate* has two offset walls with openings, *split field* has three bars between spawn and goal, and *open field* has no obstacles, $R_v = 5.6$ and faster sheep.

V. METHOD

A. Reward

Let c_t be the flock centroid, $D = 20\sqrt{2}$, H_t the convex-hull area of the flock, δ_t and m_t the mean and maximum sheep-to-goal distance, and ν_t the visible fraction. The reward is

$$r_t = \left(1 - \frac{\|c_t - g\|}{D}\right) - 0.1 \frac{H_t}{400} - 0.5 \mathbb{1}[\|d_t - g\| < \|c_t - g\|] \\ + 0.3 \left(1 - \frac{\|d_t - c_t\|}{D}\right) + 3.0(\delta_{t-1} - \delta_t) + 2.5(m_{t-1} - m_t) \\ + 0.8(\nu_t - \nu_{t-1}) - 0.05(1 - \nu_t) - 0.25 \mathbb{1}[\nu_t = 0] \\ - 0.05 n_t^{\text{stray}} + 0.35 \left(1 - \frac{\|d_t - p_t\|}{D}\right) + r_t^{\text{coll}}.$$

TABLE I

SUCCESS RATE OF EACH VERSION ON ITS OWN TRAINING SETTING AND UNDER A SHIFT: A UNIFORMLY DRAWN GOAL FOR v1 AND v2 (150 EPISODES; RANDOM-POLICY SUCCESS IN PARENTHESES), AND THE HELD-OUT SPLIT AND OPEN FIELDS FOR v3. STRUCTURED v3 COMES FROM ITS ORIGINAL SMALL EVALUATION; PROCEDURAL v3 GIVES THE RANGE OF THE TWO CURRICULA’S THREE-SEED MEANS.

Version	Change	Own	Shifted
v1	Full visibility, fixed goal	99%	2% (4%)
v2	Visibility 8, two obstacles	69%	3% (13%)
Structured v3	Recurrent, 4 layouts	75%	0% / 0%
Procedural v3	Procedural layouts	n/a	12 to 28% / 21 to 28%

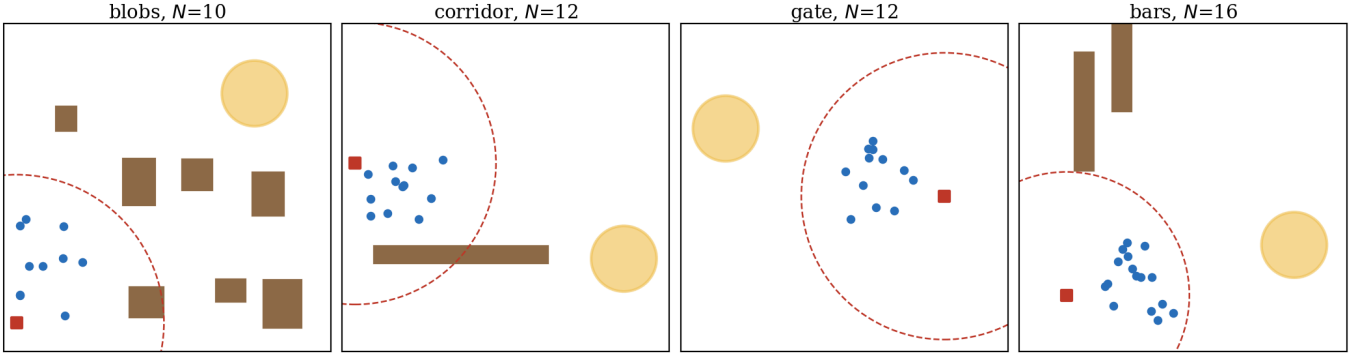


Fig. 1. Samples from the held-out procedural test distribution. Red square: dog. Dashed circle: visibility radius. Blue: sheep. Yellow: goal region. N is the flock size.

The terms reward a centroid near the goal, a compact and visible flock, and progress (as potential differences [21]), and penalize a dog between flock and goal. n_t^{stray} counts sheep more than 1.8ρ from the centroid, and p_t is a drive point behind the centroid on the goal axis at distance $\min(0.7R_v, 0.9R_f)$, where a collect-and-drive dog would stand [2]. r_t^{coll} is -0.2 on first contact with an obstacle or wall and -0.03 per step while contact lasts, and success adds $+20$.

B. Recurrent PPO

We use recurrent PPO [22] with an LSTM actor-critic of hidden size 256 from `sb3-contrib` [23]. Each update collects 256 steps from 12 parallel environments and trains for 10 epochs with batch size 384, $\gamma = 0.99$, GAE $\lambda = 0.95$, clip 0.2 and entropy coefficient 0.005. The learning rate decays linearly from 3×10^{-4} to zero over 6×10^5 steps, and observations and rewards are normalized with running statistics. Every 25k steps the policy runs 30 validation episodes, and the best checkpoint is kept with its normalization statistics. A run takes 70 to 85 minutes on a 16-core CPU.

C. Adaptive curriculum

The curriculum has stages $\sigma \in \{0, 0.33, 0.66, 1\}$. Stage 0 uses open, blob and bar layouts and shrinks every randomized range to 25% of its width; stage 0.33 adds gates and widens ranges to 60%; stage 0.66 adds corridors, uses the full ranges and randomizes the flock size. Stage 1 has the same distribution with stricter thresholds, and the goal is random at every stage. A callback promotes the agent when the last 40 training episodes meet the next stage’s thresholds on the fraction of the flock at the goal, visibility, collision events and progress. The base setting requires 30%, 55% and 70% of the flock at the goal for stages 0.33, 0.66 and 1. After any change the stage is fixed for 25k steps, a demotion requires missing a threshold by 25% of its value, and collisions can block a promotion but never cause a demotion. Our ablation, *gate 0.45*, lowers the stage-0.66 threshold from 0.55 to 0.45, which moves its demotion bound from 0.41 to 0.34 and changes nothing else.

D. Baselines

Heuristic. When at least four sheep are visible and the two furthest apart are more than 2 units apart, the expert splits the

visible sheep into two clusters and works on the one further from the goal. If a sheep in that cluster is more than 3.2 units from the cluster centroid, the dog moves behind it to collect it; otherwise it moves to a drive point behind the centroid. An obstacle-avoidance term is added, and with no sheep visible it returns to the last seen centroid or searches toward the goal. Unlike the learned agents, it reads the dog’s absolute position and every obstacle regardless of visibility, and it uses the nominal visibility and flee radii.

Behavioral cloning (BC). A random forest [24] with 300 trees of depth 18 regresses the expert’s action from raw coordinates and 15 geometric summaries of the visible flock. On held-out episodes it reaches R^2 of 0.59 and 0.69 on the two action components and a mean heading error of 29° . Its demonstrations came from an earlier environment with goals in the far quadrant and are no longer available, so the clone tests cloning from a narrower distribution, not cloning in general.

VI. EXPERIMENTAL SETUP

Each agent plays the procedural test split and the five maps, 150 episodes per scenario (100 for BC), with deterministic actions. Episode seeds start at 100000 for every agent, so all agents see the same episodes. The heuristic and BC read an older observation format with exactly ten sheep slots, so for matched comparisons we also fix the flock at ten sheep for PPO and report the randomized flock separately. We train three seeds (0, 1, 2) of each curriculum setting and evaluate each run’s selected checkpoint. The main metric is the fraction of the flock inside the goal at the end of the episode (FracGoal), which gives partial credit; we also report success rate (SR, all sheep inside). Intervals are percentile bootstrap intervals over episodes with 2000 resamples [17].

VII. RESULTS

A. Comparison with the baselines

Table II and Fig. 2 give the held-out results. On the procedural test the heuristic delivers 47% of the flock and succeeds in 35% of episodes. PPO delivers 30% with the base curriculum and 32% with *gate 0.45*, and every one of the

TABLE II

HELD-OUT RESULTS WITH A 10-SHEEP FLOCK. BASELINES: FRACGOAL WITH 95% BOOTSTRAP INTERVAL, AND SR. PPO: MEAN OVER THREE SEEDS WITH THE LOWEST AND HIGHEST SEED IN PARENTHESES, AND MEAN SR. THE LAST ROWS GIVE THE PROCEDURAL TEST WITH 8 TO 16 SHEEP AND THE MEAN OVER THE SIX SCENARIOS.

Scenario	Heuristic		Behavioral cloning		PPO, base curriculum		PPO, gate 0.45	
	FracGoal	SR	FracGoal	SR	FracGoal	SR	FracGoal	SR
Procedural test	0.468 [0.395, 0.537]	0.35	0.010 [0.001, 0.024]	0.00	0.296 (0.261, 0.326)	0.20	0.321 (0.279, 0.358)	0.19
Split field	0.317 [0.276, 0.360]	0.08	0.196 [0.161, 0.233]	0.01	0.424 (0.184, 0.576)	0.28	0.305 (0.218, 0.356)	0.12
Dense	0.269 [0.207, 0.333]	0.19	0.062 [0.036, 0.090]	0.00	0.205 (0.040, 0.289)	0.06	0.322 (0.179, 0.401)	0.13
Open field	0.989 [0.975, 1.000]	0.98	0.599 [0.519, 0.674]	0.39	0.370 (0.233, 0.565)	0.21	0.482 (0.346, 0.627)	0.28
Corridor	0.000 [0.000, 0.000]	0.00	0.008 [0.000, 0.023]	0.00	0.004 (0.000, 0.007)	0.00	0.040 (0.021, 0.064)	0.02
Narrow gate	0.003 [0.000, 0.007]	0.00	0.021 [0.008, 0.036]	0.00	0.005 (0.000, 0.011)	0.00	0.062 (0.052, 0.082)	0.02
Test, 8 to 16 sheep					0.234 (0.209, 0.281)	0.13	0.289 (0.279, 0.305)	0.16
Mean of six	0.341		0.149		0.217		0.255	

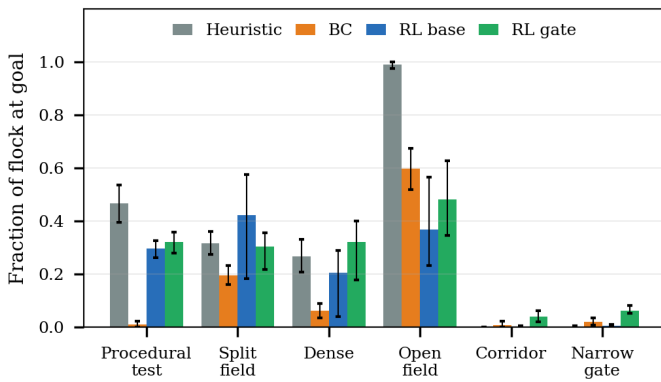


Fig. 2. FracGoal on the held-out scenarios. Baseline error bars: 95% bootstrap intervals. PPO bars: mean of three seeds, error bars span the lowest and highest seed.

six runs falls below the heuristic’s interval. The heuristic is also far ahead on the open field (99%), where its rule was designed to work.

The learned policies are ahead of the heuristic on particular maps, and the curriculum decides which. With the base curriculum, two of three seeds deliver 58% and 51% of the split field against the heuristic’s 32%, with non-overlapping intervals; the heuristic works on one cluster at a time and runs out of time in 92% of its split-field episodes. With gate 0.45, two seeds deliver 39% and 40% of the dense map against the heuristic’s 27%, although the third delivers 18%. BC is the weakest agent almost everywhere. It keeps some competence on the open field (60%), the setting closest to its training data, and falls to 1% on the procedural test, as expected when a 29° heading error compounds in closed loop [20]. No agent handles the corridor or the narrow gate, which require pushing the whole flock through an opening.

B. Seed variance

On the procedural test, which averages over many layouts, the three seeds of each setting lie within 8 points of each other. On single maps they differ by up to 39 points: base seeds deliver between 18% and 58% of the split field. Base seed 2 did best on the procedural test and worst on three maps,

and its checkpoint came from 175k steps, early in training. Had we trained only seed 0, we would have reported 45% and 49% success on the split and open fields; the three-seed means are 28% and 21%. Even without changing the configuration, runs differ: two early runs that differed only in a threshold that did not yet bind reached training FracGoal of 0.52 and 0.30 at 75k steps, because parallel environments are not bit-reproducible.

C. Curriculum ablation

With the base curriculum no seed reached stage 1, and each demotion from stage 0.66 came from delivery: at full difficulty the policy could not keep FracGoal above the demotion bound of 0.41. The runs spent only 13%, 28% and 39% of training at stage 0.66, so most learning happened without corridors or varied flock sizes. Gate 0.45 raised that share to 43%, 50% and 55% (Fig. 3). Two seeds then held stage 0.66 for 167k and 212k consecutive steps, while seed 0 still dropped back every 25k to 40k steps, because the 40-episode window dips below the bound right after each promotion. All three gate checkpoints came from late in training (425k to 575k steps), and best validation scores were similar (0.43 mean versus 0.41).

On held-out scenarios, gate 0.45 raised the mean on five of six (Table II). The gains are largest where the base runs had little training coverage. Every gate seed delivers some of the flock through corridors and narrow gates (2% to 8%, intervals excluding zero), while every base seed stays near 1% or below. The randomized-flock test rises from 23% to 29%, and the open field from 37% to 48%. The split field falls from 42% to 31%, and none of the gate seeds beats the heuristic there. Stage 0.33 draws bar layouts more often than full difficulty does (25% against 20%), so moving training time to stage 0.66 trades some bar-specific skill for coverage of corridors, gates and flock sizes. The change on the procedural test itself (30% to 32%) is within the seed spread of both settings.

D. Delivered but not held

The final fraction of the flock at the goal hides how close the learned policies get. On the split, dense and open maps,

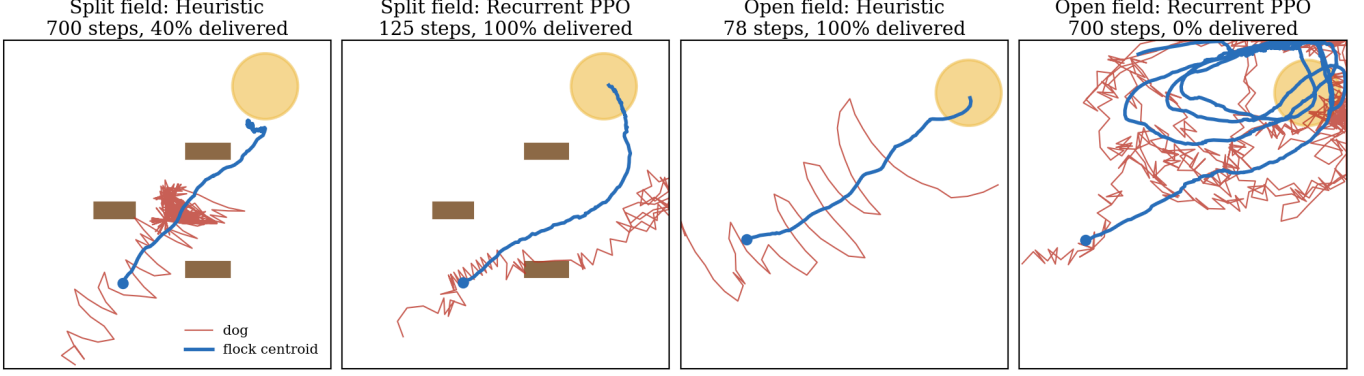


Fig. 4. Dog (red) and flock-centroid (blue, dot at start) paths for the heuristic and base PPO seed 0 on the same episodes. For each map we show the first episode in which the two agents’ outcomes differ, so the panels illustrate behavior and are not typical outcomes.

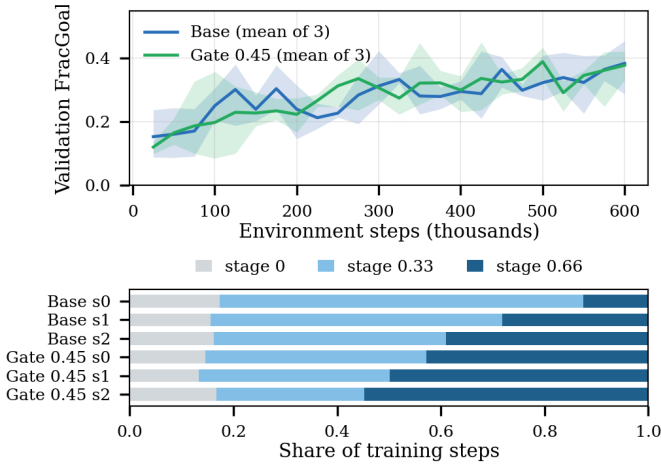


Fig. 3. Top: validation FracGoal (30 episodes every 25k steps), mean and range over three seeds per curriculum. Bottom: share of training spent at each stage by each run; no run reached stage 1.

the largest fraction of the flock inside the goal during an episode averages 51% to 79% for PPO, but the final fraction is 21% to 48%, a loss of 27 to 37 points. In 22% to 31% of these episodes at least 80% of the flock reached the goal and less than half remained at the end. The heuristic loses 1 point on the open field and 13 to 15 points on the other two maps. Fig. 4 shows the pattern: on the open field the learned dog brings the flock to the goal, then keeps circling and drives it back out. Gate 0.45 reaches the goal more often than the base curriculum but holds the flock no better. We think the cause is the reward. Success requires every sheep inside at once, the progress terms stop paying once sheep are inside, and the drive term keeps pulling the dog to the far side of the flock, which pushes sheep through the goal region.

E. Training stability

Four earlier runs failed or were stopped, each for a different reason (Table III). Two of the causes are easy to miss. Arena walls count as collisions and random goals often lie near a wall, so a collision threshold calibrated with a corner goal blocked the curriculum. And each promotion adds harder layouts, which raised collisions enough to demote the agent

TABLE III
FAILURE MODES FOUND IN SUCCESSIVE TRAINING RUNS AND THE CHANGE THAT REMOVED EACH.

Run	Symptom	Fix
1	Validation FracGoal 0 for 433k steps; stage changes every rollout	Random goal from stage 0; 25k-step stage lock and 25% demotion margin
2	Stuck at stage 0.33 by wall contacts	Collision thresholds 14/11/9 raised to 22/19/16
3	Demoted after each promotion as collisions rise	Collisions block promotion only
4	Checkpoint lacks matching normalization; noisy selection; late drift	Save statistics with checkpoint; 30 validation episodes; linear learning-rate decay

while delivery was still improving; once collisions stopped causing demotions, they fell below the old thresholds on their own, because the reward still penalizes them.

VIII. DISCUSSION

Each earlier version failed on the first variation it had not trained on, and the procedural version is the first whose every run transfers to some degree to maps it never saw. Training coverage mattered more than the architecture: the same recurrent policy class failed on held-out maps after training on four layouts and did not after training on procedural ones. The curriculum ablation shows the same effect at a smaller scale, since longer training at full difficulty helped most on the corridor and gate layouts that only appear there. It also shows a trade-off: the base setting produced the only policies ahead of the heuristic on the split field, while gate 0.45 produced policies ahead on the dense map and more consistent across seeds (largest spread 28 points against 39). Three seeds per setting show the large differences but not the 2-point change on the procedural test.

The heuristic still leads on the procedural test and in open space. Three factors likely contribute: it reads the full obstacle map, while the policy has 24 lidar rays; the policy never trained at stage 1 or on the test’s wider ranges; and the policy does not hold a delivered flock. The last factor suggests the

most direct next step, a reward for keeping sheep inside the goal or an episode that ends once most of the flock is inside. Other next steps are a residual policy on top of the heuristic, which would keep its open-field competence, and a clone retrained on demonstrations from the procedural distribution.

For evaluation, a single run scored on a few hand-designed maps would have supported a much stronger claim than six runs scored on a procedural test suite, which agrees with the advice to report many layouts and several seeds [6], [16].

Limitations. Three seeds per setting show the variance but estimate it poorly. The environment is a 2-D kinematic simulation with rectangular obstacles, and obstacles do not block sight. The five maps are single layouts and serve as case studies. The heuristic’s parameters were not retuned for them, and the heuristic has inputs the learned policy lacks. The structured-v3 numbers come from a small evaluation of a model we could not re-evaluate. We control a single herder; cooperation between herders [10], [11] is out of scope.

IX. CONCLUSION

Four versions of a learned herder show that each fixed training setting produced a policy that failed as soon as the setting changed, from a fixed goal (99% to 2% success) to fixed layouts (75% to 0%). Training recurrent PPO on procedural layouts with an adaptive curriculum gives policies that transfer to held-out maps and are ahead of a collect-and-drive heuristic on some of them, though not on the procedural test suite as a whole (32% against 47% of the flock delivered). Which maps the learned policy wins depends on the curriculum, and single-map results vary so much across seeds that one run would have overstated them. The largest remaining gap is holding the flock: on the hand-designed maps the policies lose 27 to 37 points of the flock after bringing it to the goal.

REFERENCES

- [1] J.-M. Lien, O. B. Bayazit, R. T. Sowell, S. Rodriguez, and N. M. Amato, “Shepherding Behaviors,” in *Proceedings of the IEEE International Conference on Robotics and Automation*, 2004, pp. 4159–4164. doi: 10.1109/ROBOT.2004.1308924.
- [2] D. Strömbom *et al.*, “Solving the Shepherding Problem: Heuristics for Herding Autonomous, Interacting Agents,” *Journal of the Royal Society Interface*, vol. 11, no. 100, p. 20140719, 2014, doi: 10.1098/rsif.2014.0719.
- [3] A. J. King *et al.*, “Biologically Inspired Herding of Animal Groups by Robots,” *Methods in Ecology and Evolution*, vol. 14, no. 2, pp. 478–486, 2023, doi: 10.1111/2041-210X.14081.
- [4] R. Vaughan, N. Sumpter, J. Henderson, A. Frost, and S. Cameron, “Experiments in Automatic Flock Control,” *Robotics and Autonomous Systems*, vol. 31, no. 1–2, pp. 109–117, 2000, doi: 10.1016/S0921-8890(99)00084-6.
- [5] A. Pierson and M. Schwager, “Controlling Noncooperative Herds with Robotic Herders,” *IEEE Transactions on Robotics*, vol. 34, no. 2, pp. 517–525, 2018, doi: 10.1109/TRO.2017.2776308.
- [6] K. Cobbe, O. Klimov, C. Hesse, T. Kim, and J. Schulman, “Quantifying Generalization in Reinforcement Learning,” in *Proceedings of the International Conference on Machine Learning (ICML)*, 2019, pp. 1282–1289.
- [7] C. W. Reynolds, “Flocks, Herds and Schools: A Distributed Behavioral Model,” in *Proceedings of the 14th Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH)*, 1987, pp. 25–34. doi: 10.1145/37401.37406.
- [8] D. Strömbom and A. J. King, “Robot Collection and Transport of Objects: A Biomimetic Process,” *Frontiers in Robotics and AI*, vol. 5, p. 48, 2018, doi: 10.3389/frobt.2018.00048.
- [9] A. Hussein, E. Petraki, S. Elsayah, and H. A. Abbass, “Autonomous Swarm Shepherding Using Curriculum-Based Reinforcement Learning,” in *Proceedings of the 21st International Conference on Autonomous Agents and Multiagent Systems (AAMAS)*, 2022, pp. 633–641.
- [10] Y. Hasan, J. E. G. Baxter, C. A. S. Castillo, E. Delgado, and L. Tapia, “Flock Navigation by Coordinated Shepherds via Reinforcement Learning,” in *Proceedings of the 15th International Workshop on the Algorithmic Foundations of Robotics (WAFR)*, 2022.
- [11] I. Napolitano, A. Lama, F. De Lellis, and M. di Bernardo, “Emergent Cooperative Strategies for Multi-Agent Shepherding via Reinforcement Learning,” *arXiv preprint arXiv:2411.05454*, 2024.
- [12] M. Hausknecht and P. Stone, “Deep Recurrent Q-Learning for Partially Observable MDPs,” in *AAAI Fall Symposium on Sequential Decision Making for Intelligent Agents*, 2015.
- [13] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997, doi: 10.1162/neco.1997.9.8.1735.
- [14] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, “Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World,” in *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2017, pp. 23–30. doi: 10.1109/IROS.2017.8202133.
- [15] S. Narvekar, B. Peng, M. Leonetti, J. Sinapov, M. E. Taylor, and P. Stone, “Curriculum Learning for Reinforcement Learning Domains: A Framework and Survey,” *Journal of Machine Learning Research*, vol. 21, no. 181, pp. 1–50, 2020.
- [16] R. Agarwal, M. Schwarzer, P. S. Castro, A. Courville, and M. G. Bellemare, “Deep Reinforcement Learning at the Edge of the Statistical Precipice,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2021, pp. 29304–29320.
- [17] B. Efron and R. J. Tibshirani, *An Introduction to the Bootstrap*. Chapman, Hall/CRC, 1994.
- [18] D. A. Pomerleau, “ALVINN: An Autonomous Land Vehicle in a Neural Network,” in *Advances in Neural Information Processing Systems*, 1989.
- [19] A. Hussein, M. M. Gaber, E. Elyan, and C. Jayne, “Imitation Learning: A Survey of Learning Methods,” *ACM Computing Surveys*, vol. 50, no. 2, pp. 1–35, 2017, doi: 10.1145/3054912.
- [20] S. Ross, G. Gordon, and D. Bagnell, “A Reduction of Imitation Learning and Structured Prediction to No-Regret Online Learning,” in *Proceedings of the International Conference on Artificial Intelligence and Statistics (AISTATS)*, 2011, pp. 627–635.
- [21] A. Y. Ng, D. Harada, and S. Russell, “Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping,” *Proceedings of the 16th International Conference on Machine Learning*, pp. 278–287, 1999.
- [22] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal Policy Optimization Algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [23] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, “Stable-Baselines3: Reliable Reinforcement Learning Implementations,” *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021.
- [24] L. Breiman, “Random Forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001, doi: 10.1023/A:1010933404324.